

Frontend Interview Questions and Answers

JavaScript Fundamentals

JS-1. Can you explain Babel and its purpose?

Babel is a JavaScript compiler that transpiles modern JavaScript (ES6+) into backward-compatible code for older browsers. Its purpose is to ensure cross-browser compatibility by converting new syntax (e.g., arrow functions, destructuring) into ES5 or other compatible formats.

JS-2. What is `eval` in JavaScript? What is its main use?

`eval` is a JavaScript function that executes a string as JavaScript code. Its main use is to dynamically execute code at runtime, but it's rarely used due to security risks (e.g., code injection) and performance issues. Safer alternatives like `Function` constructors are preferred.

JS-3. What is the event loop? How does it work?

The event loop is a mechanism in JavaScript's runtime that handles asynchronous operations. It continuously checks the call stack and task queue. When the stack is empty, it takes tasks from the queue (e.g., `setTimeout` callbacks) or microtask queue (e.g., Promises) and executes them. Microtasks have higher priority than macrotasks.

JS-4. What will be the output?

```
abc()
function abc() {
  console.log("1")
}
abc()
function abc() {
  console.log("2")
}
abc()
function abc() {
  console.log("3")
}
var abc = function () {
  console.log("4")
}
abc()
var abc = function () {
  console.log("5")
}
abc()
var abc = function () {
  console.log("6")
}
abc()
```

Output: 3, 3, 3, 4, 5, 6

Explanation: Function declarations are hoisted, and the last one (`console.log("3")`) overwrites previous ones. The first three `abc()` calls output 3. The `var` assignments create new function expressions, overwriting `abc` each time, so the last three calls output 4, 5, and 6.

JS-5. What will be the output?

```
console.log("1");
setTimeout(() => console.log("2"), 0)
Promise.resolve().then(() => console.log("3"))
console.log("4");
```

Output: 1, 4, 3, 2

Explanation: Synchronous code runs first (1, 4). Microtasks (Promise `.then`) run next (3). Macrotasks (`setTimeout`) run last (2).

JS-6. What is the difference between `let` vs `const` vs `var`?

- `var`: Function-scoped, hoisted with `undefined`, allows redeclaration and reassignment.
- `let`: Block-scoped, hoisted but not initialized (Temporal Dead Zone), allows reassignment, no redeclaration.
- `const`: Block-scoped, hoisted but not initialized, cannot be reassigned or redeclared (object properties can be mutated).

JS-7. What is the Temporal Dead Zone in JavaScript?

The Temporal Dead Zone (TDZ) is the period between a block's start and a `let` or `const` variable's declaration, where accessing the variable throws a `ReferenceError` due to hoisting without initialization.

JS-8. What is the difference between `call` and `apply`?

- `call`: Invokes a function with a specified `this` context and arguments passed individually. Example: `fn.call(thisArg, arg1, arg2)`.
- `apply`: Invokes a function with a specified `this` context and arguments passed as an array. Example: `fn.apply(thisArg, [arg1, arg2])`.

Promises and Polyfills

PP-1. Can you explain `Promise.all`, `Promise.race`, `Promise.allSettled` and the differences?

- `Promise.all`: Resolves when all promises resolve, rejects if any reject, returning an array of results.
- `Promise.race`: Resolves or rejects with the first promise that settles.
- `Promise.allSettled`: Resolves when all promises settle (resolved or rejected), returning an array of objects with `status` (`fulfilled` or `rejected`) and `value` or `reason`.

Differences:

- `Promise.all` fails fast on rejection.
- `Promise.race` returns the first settled promise.
- `Promise.allSettled` waits for all promises, capturing all outcomes.

PP-2. Write a polyfill for `Promise.all` function

```
function promiseAll(promises) {
  return new Promise((resolve, reject) => {
    if (!Array.isArray(promises)) {
      return reject(new TypeError('Promises must be an array'));
    }
    const results = [];
    let resolvedCount = 0;
    if (promises.length === 0) return resolve(results);

    promises.forEach((promise, index) => {
      Promise.resolve(promise)
        .then(value => {
          results[index] = value;
          resolvedCount++;
          if (resolvedCount === promises.length) resolve(results);
        })
        .catch(reject);
    });
  });
}
```

PP-3. Write a polyfill for `Array.map` function

```
Array.prototype.myMap = function(callback, thisArg) {
  const result = [];
  for (let i = 0; i < this.length; i++) {
    if (i in this) {
      result[i] = callback.call(thisArg, this[i], i, this);
    }
  }
  return result;
};
```

Web Performance and Optimization

WPO-1. What are Core Web Vitals? Why and when do we use them?

Core Web Vitals are metrics to measure user experience:

- **LCP (Largest Contentful Paint):** Time to render the largest content element.
- **FID (First Input Delay):** Time from user interaction to browser response.

- **CLS (Cumulative Layout Shift):** Visual stability during page load.

Why/When: Used to optimize user experience, improve SEO, and meet Google's ranking criteria. Measure during development and production to ensure performance.

WPO-2. What is Cumulative Layout Shift (CLS)? What does it impact, and how to fix it?

CLS measures visual stability by quantifying unexpected layout shifts during page load.

Impact: Poor CLS frustrates users (e.g., clicking wrong elements) and affects SEO.

Fixes:

- Set explicit dimensions for images/videos (**width, height**).
- Avoid dynamically inserted content without reserving space.
- Use CSS **transform** for animations instead of properties that trigger layout changes.

WPO-3. Can you explain more about FCP and LCP?

- **FCP (First Contentful Paint):** Time from navigation to when the first DOM content (e.g., text, image) is rendered. Indicates initial page load speed.
- **LCP (Largest Contentful Paint):** Time to render the largest visible content element (e.g., hero image, heading). Reflects main content load time.
Use: Optimize FCP with server-side rendering or faster assets; optimize LCP by prioritizing critical content and reducing render-blocking resources.

WPO-4. What is the optimized way to add an **onclick** event to a 10-column, 100-row table without adding **onclick** to each element?

Use **event delegation**:

- Add a single **onclick** event listener to the table's parent element.
- Check the **event.target** to identify the clicked cell/row.

Example:

```
document.querySelector('table').addEventListener('click', (event) => {
  const cell = event.target.closest('td');
  if (cell) {
    const row = cell.parentElement.rowIndex;
    const col = cell.cellIndex;
    console.log(`Clicked row ${row}, column ${col}`);
  }
});
```

Benefits: Reduces memory usage and simplifies event management.

WPO-5. How do you monitor application performance from the end user?

- Use tools like **Lighthouse**, **Google PageSpeed Insights**, or **Web Vitals** to measure Core Web Vitals.
- Implement **Real User Monitoring (RUM)** with tools like Sentry, New Relic, or custom analytics to track performance metrics (e.g., LCP, FID) in production.

- Analyze browser performance APIs (**PerformanceObserver**, **Navigation Timing API**) for real-time data.

WPO-6. When an application is too slow on initial page load, how do you approach and optimize it?

1. **Diagnose:** Use Lighthouse or Chrome DevTools to identify bottlenecks (e.g., render-blocking resources, large assets).
2. **Optimize:**
 - Minify CSS/JS, use code splitting, and tree shaking.
 - Lazy-load images and non-critical assets.
 - Use SSR/SSG for faster initial rendering.
 - Optimize server response time (e.g., CDN, caching).
 - Compress assets (e.g., WebP for images).
3. **Monitor:** Continuously track performance with RUM tools.

Microfrontends

MF-1. What is the difference between monorepo vs microfrontend?

- **Monorepo:** A single repository containing multiple projects or modules, sharing code and dependencies. Simplifies dependency management and code reuse.
 - **Microfrontend:** An architecture splitting a frontend into independently deployable, loosely coupled modules, each owned by a team. Enables scalability but increases complexity.
- Key Difference:** Monorepo is about repository structure; microfrontend is about application architecture.

MF-2. What are the ways to communicate between microfrontends?

- **Custom Events:** Dispatch and listen to events via **window.dispatchEvent** and **window.addEventListener**.
- **Shared State:** Use a global store (e.g., Redux, Zustand) or browser storage (**localStorage**, **sessionStorage**).
- **URL Parameters:** Pass data via query strings or URL paths.
- **PostMessage API:** Communicate between iframes or windows.
- **Pub/Sub Libraries:** Use libraries like RxJS or EventEmitter for event-based communication.

MF-3. When is the right time to choose microfrontends?

Use microfrontends when:

- Teams need independent deployment (e.g., large teams with separate domains).
 - The app has distinct modules with minimal interdependence.
 - Legacy systems need incremental modernization.
- Avoid for small apps or tightly coupled systems due to added complexity.

MF-4. In a microfrontend application with design inconsistency issues, how do you fix it?

- Adopt a **design system** with shared components and tokens (e.g., colors, typography).

- Use a shared CSS library or component library (e.g., Storybook) across microfrontends.
- Enforce style guides via linters (e.g., Stylelint).
- Implement a centralized theme provider or CSS-in-JS solution to ensure consistency.
- Regularly audit UI with tools like Chromatic or Figma plugins.

Accessibility

A11Y-1. What is accessibility in web applications? Benefits of it?

Accessibility (a11y) ensures web apps are usable by people with disabilities (e.g., screen reader support, keyboard navigation).

Benefits:

- Wider audience reach, including users with disabilities.
- Improved SEO (e.g., better semantic HTML).
- Compliance with laws (e.g., ADA, WCAG).
- Enhanced user experience for all.

A11Y-2. What is the automated way to test accessibility?

- Use tools like **Lighthouse**, **axe-core**, or **pa11y** to scan for WCAG violations.
- Integrate accessibility tests into CI/CD pipelines with libraries like **@axe-core/jest** or **cypress-axe**.
- Use browser extensions (e.g., WAVE, Accessibility Insights) for quick audits.

A11Y-3. How do you test accessibility during development?

- Use automated tools (e.g., axe-core, Lighthouse) in development workflows.
- Test with screen readers (e.g., NVDA, VoiceOver).
- Ensure keyboard navigability (e.g., **tabindex**, focus management).
- Validate semantic HTML (e.g., ARIA roles, landmarks).
- Manually test with assistive technologies and WCAG checklists.

Testing

TEST-1. What is Unit, E2E, and Integration testing?

- **Unit Testing:** Tests individual components or functions in isolation (e.g., Jest, Mocha).
- **Integration Testing:** Tests interactions between components or modules (e.g., React Testing Library).
- **E2E Testing:** Tests the entire application flow from the user's perspective (e.g., Cypress, Playwright).

Debugging and Security

DS-1. How do you debug production issues without console or API errors?

- Use **Real User Monitoring** (e.g., Sentry, LogRocket) to capture user interactions and errors.
- Check browser network tab for failed requests or slow resources.
- Reproduce the issue in a staging environment with similar conditions.
- Analyze performance metrics (e.g., Core Web Vitals) for clues.

- Enable debug logging temporarily in production (if safe).

DS-2. A customer reports they cannot access a page. What is your first action?

- **Reproduce the issue:** Ask for details (browser, device, steps) and try to replicate it.
- Check for **client-side errors** (e.g., console, network tab).
- Verify **server status** (e.g., API availability, server logs).
- Confirm **user permissions** or session issues.
- Use monitoring tools (e.g., Sentry) to identify patterns.

DS-3. How do you prevent security issues in web applications?

- Use **HTTPS** to encrypt data.
- Sanitize inputs to prevent XSS (e.g., libraries like DOMPurify).
- Implement **CSP (Content Security Policy)** to restrict resource loading.
- Use secure cookies (**HttpOnly**, **Secure**, **SameSite**).
- Validate and sanitize API inputs/outputs.
- Regularly update dependencies to patch vulnerabilities (e.g., **npm audit**).

DS-4. What is CSP?

Content Security Policy (CSP) is a security mechanism to prevent cross-site scripting (XSS) and other attacks by restricting which resources (e.g., scripts, images) can be loaded. It's defined via HTTP headers or `<meta>` tags, specifying trusted sources (e.g., `script-src 'self'`).

Browser Storage

BS-1. How do you store data in the browser?

- **localStorage:** Persistent key-value storage (~5-10MB), survives browser close.
- **sessionStorage:** Session-based key-value storage, cleared on tab close.
- **Cookies:** Small key-value pairs sent with HTTP requests, useful for authentication.
- **IndexedDB:** Structured, asynchronous storage for large data (e.g., files).
- **WebSQL:** Deprecated, not recommended.

React and State Management

RSM-1. Can you explain React patterns?

- **Component Patterns:** Functional components, Hooks, Compound Components, Render Props.
- **State Management:** Context API, Redux, Zustand for global state.
- **Performance:** `useMemo`, `useCallback`, `React.memo` to prevent unnecessary renders.
- **Controlled vs Uncontrolled:** Controlled components manage state via props; uncontrolled use refs.
- **HOC (Higher-Order Components):** Wrap components to add functionality.

RSM-2. How do you effectively maintain state management in a web application?

- Use **local state** (`useState`, `useReducer`) for component-specific data.
- Use **Context API** for small-scale shared state.
- Use libraries like **Redux**, **Zustand**, or **MobX** for complex global state.

- Minimize state updates to prevent re-renders.
- Normalize data for efficient updates and retrieval.

RSM-3. What is Context API? When do we consider it?

Context API is React's built-in solution for sharing state across components without prop drilling.

When to use:

- Small to medium-sized apps with shared state (e.g., theme, user data).
- Avoid for complex state logic (use Redux/Zustand instead).
- Use for global, infrequently updated data.

RSM-4. When do you choose a state management library over Context API?

Choose a library (e.g., Redux, Zustand) when:

- State is complex with frequent updates.
 - You need middleware for async operations (e.g., Redux Thunk).
 - Debugging requires tools like Redux DevTools.
 - Large teams need predictable state management.
- Use Context API for simple, small-scale state sharing.

RSM-5. What is the difference between `useMemo` and `React.memo`?

- `useMemo`: A Hook that memoizes a value to prevent recalculation unless dependencies change.
Example: `const memoizedValue = useMemo(() => computeExpensiveValue(a, b), [a, b])`.
- `React.memo`: A higher-order component that memoizes a component to prevent re-rendering unless props change.
Example: `const MyComponent = React.memo((props) => <div>{props.value}</div>)`.

RSM-6. How do you optimize re-renders in a React application?

- Use `React.memo` for components with stable props.
- Use `useMemo` for expensive computations.
- Use `useCallback` for functions passed as props.
- Split large components into smaller ones to isolate state changes.
- Avoid inline objects/functions in JSX to prevent prop changes.
- Use tools like React Profiler to identify unnecessary renders.

RSM-7. When do you choose React over Angular.js or Ember.js?

Choose React when:

- You need flexibility and a component-based architecture.
- The project benefits from a large ecosystem (e.g., React Native).
- You prefer functional components and Hooks over class-based frameworks.
- Fast rendering and virtual DOM are priorities.

Choose Angular/Ember for opinionated, full-featured frameworks with built-in solutions (e.g., routing, state).

RSM-8. Why choose Zustand vs Redux in a React app?

- **Zustand:** Lightweight, simple API, minimal boilerplate, Hooks-based, good for small to medium apps.
- **Redux:** Structured, predictable state management, robust middleware (e.g., Thunk), Redux DevTools, better for large, complex apps.

Choose Zustand for simplicity; Redux for scalability and debugging.

RSM-9. What is reconciliation in React?

Reconciliation is React's process of updating the DOM by comparing the new virtual DOM with the previous one (diffing). It determines the minimal changes needed to update the real DOM efficiently.

RSM-10. What is Fiber in React?

React Fiber is the reimplementation of React's core reconciliation algorithm (introduced in React 16). It enables incremental rendering, prioritization of updates, and better handling of animations and layouts by breaking rendering into smaller, interruptible units.

RSM-11. What will be the output?

```
function A() {
  console.log("A");
  return <B />;
}
function B() {
  console.log("B");
  return <C />;
}
function C() {
  console.log("C");
  return null;
}
function D() {
  console.log("D");
  return null;
}
export default function App() {
  const [state, setState] = useState(0);
  useEffect(() => {
    setState((state) => state + 1);
  }, []);
  console.log("App");
  return (
    <div>
      <A state={state} />
      <D />
    </div>
  );
}
```

Output: App, A, B, C, D, App, A, B, C, D

Explanation:

- On initial render: App logs, then A renders (A logs), calls B (B logs), which calls C (C logs), and D renders (D logs).
- `useEffect` triggers a state update (`setState`), causing a re-render, repeating the same logs: App, A, B, C, D.

Design Systems

DSYS-1. What is a design system? Benefits of it?

A design system is a collection of reusable components, styles, and guidelines (e.g., colors, typography) to ensure consistent UI/UX.

Benefits:

- Faster development with reusable components.
- Consistent design across teams/products.
- Easier maintenance and scalability.
- Improved collaboration between designers and developers.

DSYS-2. How do you design a design system supporting multiple frameworks (React, Vue, Angular)?

- Use **design tokens** (e.g., JSON for colors, sizes) to define styles agnostically.
- Create a core CSS library (e.g., Tailwind, CSS-in-JS) shared across frameworks.
- Build framework-specific component libraries (e.g., React Storybook, Vue components).
- Use tools like Storybook for cross-framework component documentation.
- Ensure accessibility and theming support (e.g., CSS variables).

DSYS-3. What is a design token?

Design tokens are the smallest pieces of a design system (e.g., colors, typography, spacing) stored in a platform-agnostic format (e.g., JSON, CSS variables). They ensure consistency across platforms and frameworks by providing a single source of truth for styling.

Module Bundling and Optimization

MBO-1. What is tree shaking? How does it benefit a web application?

Tree shaking eliminates unused code (dead code) from JavaScript bundles during the build process (e.g., via Webpack, Rollup).

Benefits:

- Smaller bundle sizes, faster load times.
- Improved runtime performance.
- Works best with ES modules (`import/export`).

MBO-2. What is a bundling tool? Why do we use it?

A bundling tool (e.g., Webpack, Rollup, Vite) combines multiple JavaScript, CSS, and other files into optimized bundles.

Why:

- Reduces HTTP requests by combining files.
- Enables tree shaking and minification.
- Handles module resolution and dependency management.
- Supports modern features (e.g., code splitting, lazy loading).

Rendering Patterns

RP-1. What are the rendering patterns in frontend?

- **Client-Side Rendering (CSR)**: Rendering in the browser using JavaScript.
- **Server-Side Rendering (SSR)**: Rendering HTML on the server before sending to the client.
- **Static Site Generation (SSG)**: Pre-rendering pages at build time.
- **Incremental Static Regeneration (ISR)**: SSG with dynamic updates.
- **Streaming SSR**: Sending partial HTML chunks to the client.

RP-2. What is the difference between CSR vs SSR vs SSG? When to choose and why?

- **CSR**: JavaScript renders content in the browser.
 - **Choose**: For dynamic, interactive apps (e.g., SPAs).
 - **Why**: Fast updates, but slow initial load and poor SEO.
- **SSR**: Server renders HTML for each request.
 - **Choose**: For SEO-critical apps or fast initial loads.
 - **Why**: Better SEO and FCP, but server-intensive.
- **SSG**: Pages pre-rendered at build time.
 - **Choose**: For static content (e.g., blogs, docs).
 - **Why**: Fastest load times, great SEO, no server runtime needed.

Miscellaneous

MISC-1. Explain the difference between shallow copy vs deep copy?

- **Shallow Copy**: Copies top-level properties, but nested objects are referenced.
Example: `const copy = {...obj}` or `Object.assign({}, obj)`.
 - **Deep Copy**: Copies all levels, creating new instances of nested objects.
Example: `JSON.parse(JSON.stringify(obj))` or libraries like Lodash `cloneDeep`.
- Difference**: Shallow copy shares references for nested objects; deep copy duplicates them.

MISC-2. How do you choose an NPM package for a web application?

- Check **popularity** (downloads, stars on GitHub).
- Verify **maintenance** (recent updates, open issues).
- Ensure **compatibility** with your framework/version.
- Assess **size** and performance impact (e.g., bundlephobia.com).
- Review **security** (e.g., `npm audit`, Snyk).
- Prefer packages with clear documentation and community support.

MISC-3. What is the difference between **debounce** vs **throttle**?

- **Debounce:** Delays function execution until after a pause in events (e.g., for search input).
Example: Only call API after user stops typing for 500ms.
 - **Throttle:** Limits function execution to once per time interval (e.g., for scroll events).
Example: Call function every 100ms during scrolling.
- Difference:** Debounce waits for inactivity; throttle ensures regular execution.

MISC-4. How do you use AI in day-to-day as a developer?

- **Code Generation:** Use tools like GitHub Copilot for boilerplate or complex logic.
 - **Debugging:** Ask AI to explain errors or suggest fixes.
 - **Learning:** Get explanations for new concepts or APIs.
 - **Automation:** Generate tests, documentation, or scripts.
 - **Optimization:** Analyze code for performance improvements.
- Note:** I can assist with code generation or debugging if needed.

MISC-5. What is the output of a shallow copy vs deep copy?

This is a clarification of MISC-1 (shallow vs deep copy). See the answer above for details.